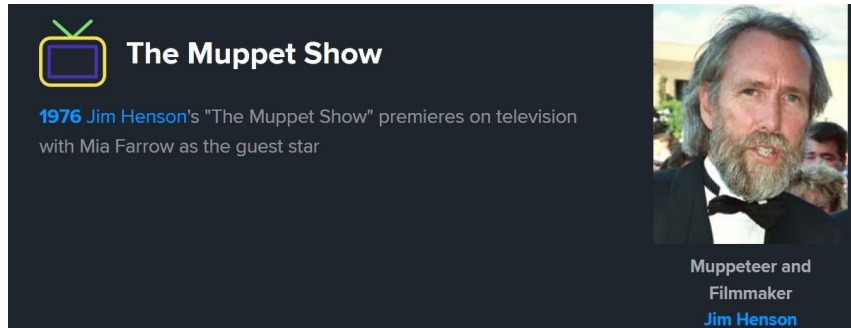


COMS W4156 Advanced Software Engineering (ASE)

September 5, 2024

On this day ...



Agenda

1. **Miniproject Overview**
2. Version Control Overview
3. Github demo: Bin Choi

Lecture Notes linked on
[Courseworks Calendar](#)



Miniproject Part 0 - read before starting work on Part 1

10: Introduction and System Setup

The miniproject is a set of individual assignments.

"Individual" means all by yourself working alone.

**Do not work with your team or other students,
do not work with an AI programming assistant,
do not outsource to a freelance developer, etc.**

If you have questions, post to [edstem](#) "staff"

There are two versions of the miniproject, Java and C++, pick one. If you do not know Java or C++ well enough to do the miniproject, you should drop the course. If you know both Java and C++, might be better to choose the Java version as it has been more thoroughly vetted



Asking Questions

We welcome clarification questions. Please ask publicly (in class or on edstem) so all students in the class can benefit from our answer. But...



One of the background requirements that you agreed to is:

"Search for and read (or watch) documentation and tutorials for software development tools, frameworks, APIs, libraries, etc. on your own, without instruction or assistance from the teaching staff."

So you should not need to ask us questions like
How do you run [software development tool]?

But if you've read/watched the docs, and you're still stuck or got some strange response, then indeed do ask us. Make sure to say you already read/watched [link to something]

Miniproject Part 1 - September 13

11: Setup, Testing and Bug Fixing

This assignment is intentionally due on program change date. If you find this assignment too challenging, you should consider dropping the course since later assignments will be much harder



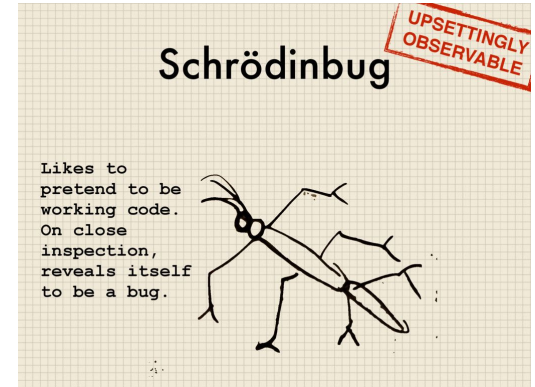
You are provided with the initial version of a simple service. Run a style checker and clean up the bad code it finds. *We expect you to find bad code because we put it there.* This process does not find/fix bugs, but makes the code more readable and thus less likely to be edited into bugginess in the future. You will also need to do style checking for the team project

Miniproject Part 1 - September 13

Flesh out the initial test suite to increase how much of the code it actually executes by following both sides of branches (more on testing "coverage" later in course), and use the tests to find bugs

You should also try to find bugs using a static analysis "bug finder" (not the same as style checking, although the same tool may do both). *We expect you to find bugs because we put them there.* Think of this process as a scavenger hunt where you devise your own clues from collecting forensic evidence

Then fix all the bugs you found, and re-run all the checks to make sure you really fixed the bugs without introducing new flaws. You will also need to find and fix bugs for the team project, but in that case you or your team members put them there



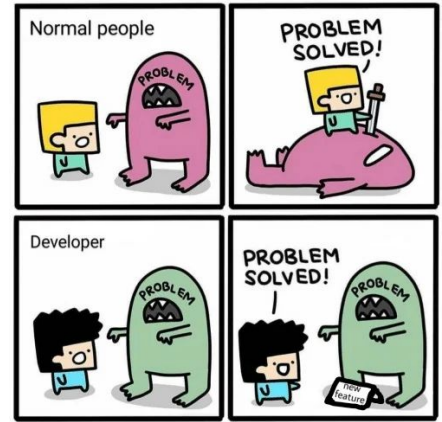
Miniproject Part 2 - September 20

I2: Feature Implementation and Maintenance

Add two new features (that we define) to the initial service, with corresponding test cases, reusing existing code when you can. Also write documentation for the new code

Run style checking, bug finding, and your expanded test suite to make sure your new features work and do not break the old code. *Although we gave you badly written buggy code to start with, you are supposed to give us clean correct code*

You will also need to develop and debug features for the team project



When Debugging fails

Miniproject Part 3 - September 27

I3: Continuous Integration and Cloud Deployment

Add CI (automatic build and test) and deploy to Google Cloud Platform using github actions. There will be examples and a demo showing how to do this

We will get GCP credits for you after the first miniproject assignment has been graded - since by then, hopefully, we will know who is really taking the course

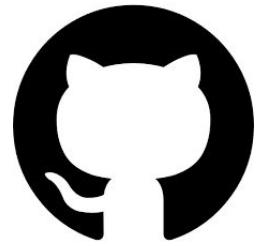


Agenda

1. Miniproject Overview
2. **Version Control Overview**
3. Github demo: Bin Choi



This Course Assumes Everyone Uses Github



Nearly everyone enrolled in the class claims that they fulfill the background requirement "Maintain your own personal codebase(s) on github or similar version control repository."

So this will be a very quick overview, followed by a more detailed demo

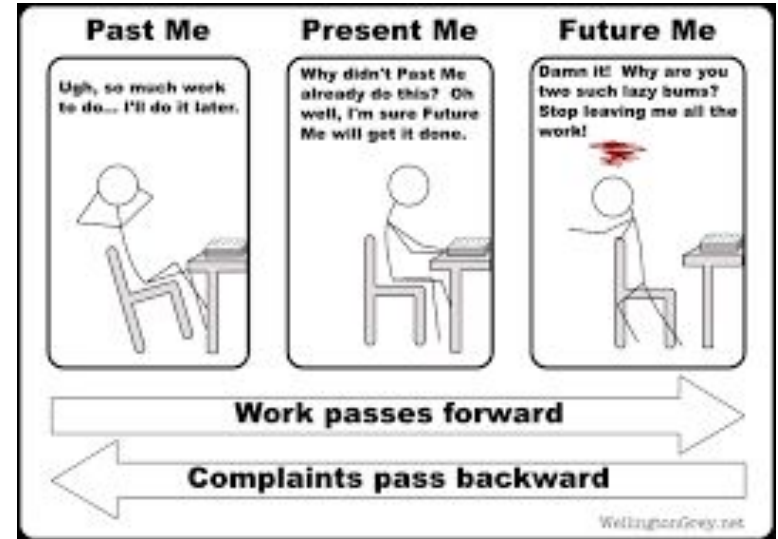


Why Use Git? (Or Any Other Version Control)

Software Engineers always work in Teams

People who write run-once or short-lived programs are programmers, not software engineers

Software engineering teams for longer lived programs always have at least three members: you, the past you, and the future you



You / Past You / Future You

Motivate...

version control - It worked last week, can I undo my changes that broke it?

coding style - Why did I name this function 'FixThisLater' two years ago?

code documentation - What does 'FixThisLater' do?

task board - Where is the sticky note with my to-do list?

and other topics that will be discussed later in course

....even if your team never grows



Do Any Real-World Developers Work Alone?

Well-known software originally developed and maintained by one person:

- Linux kernel (Linus Torvalds)
- Java compiler and JVM (James Gosling)
- Facebook (Mark Zuckerberg)



Multi-Member Teams

Software engineering teams usually have more members, all of whom join the team at some point (not necessarily beginning) and may leave at some point (not necessarily end)

A team of 4-5 software engineers often has several other members (sometimes shared with other teams), such as tech lead, engineering manager, product manager, tech writer, customer support and/or devops liaison, UIUX designer and/or researcher (when applicable), possibly a [separate testing team](#), with several layers of management above each

Some teams also have student interns or trainees

It takes all four of these people to make a good team



Multi-Member Engineering Teams

Mandate...

version control - My code worked last week, now it doesn't, who broke it?

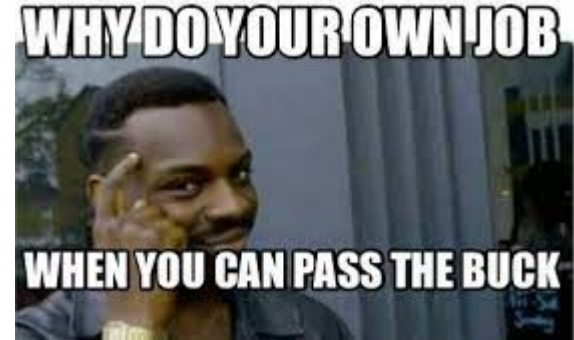
coding style - Why did Jing, who left two years ago to work for Kookle, name this function 'FixThisLater'?

code documentation - What does 'FixThisLater' do?

task board - What am I supposed to be working on?

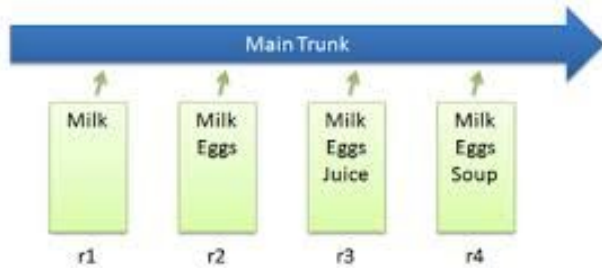
and other topics that will be discussed later in course

...particularly as team grows or churns

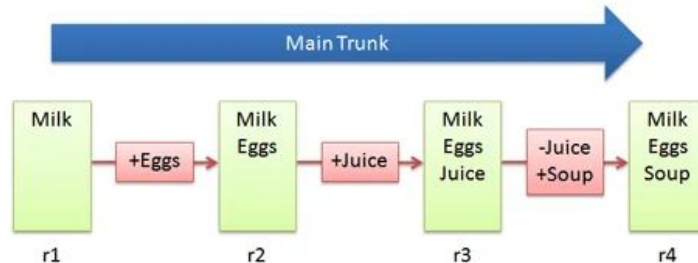


"A Visual Guide To Version Control"

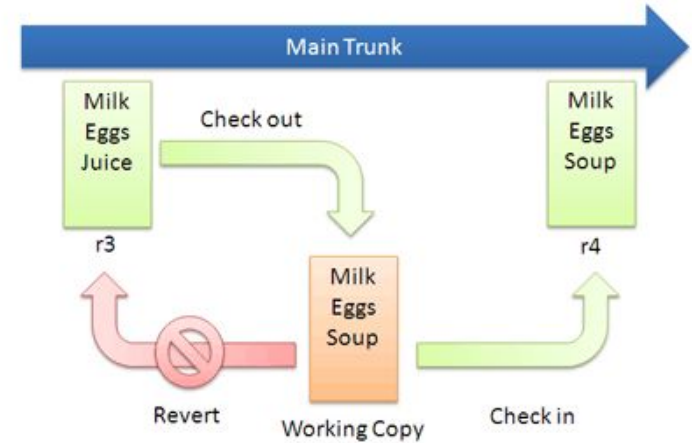
Basic Checkins



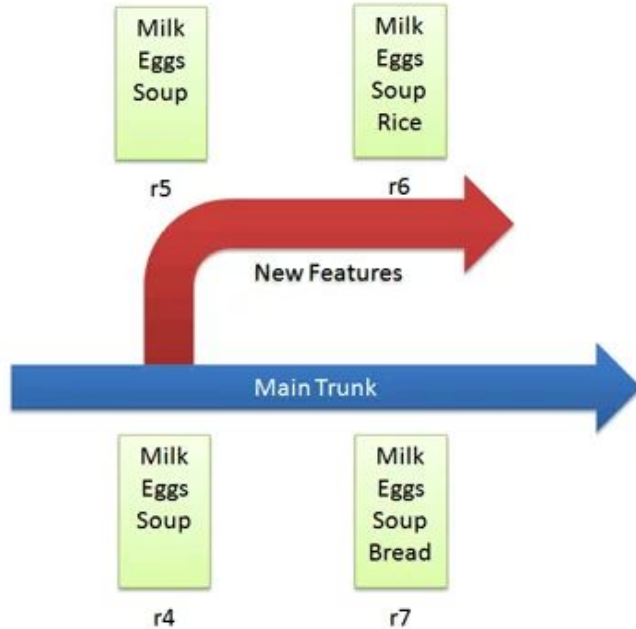
Basic Diffs



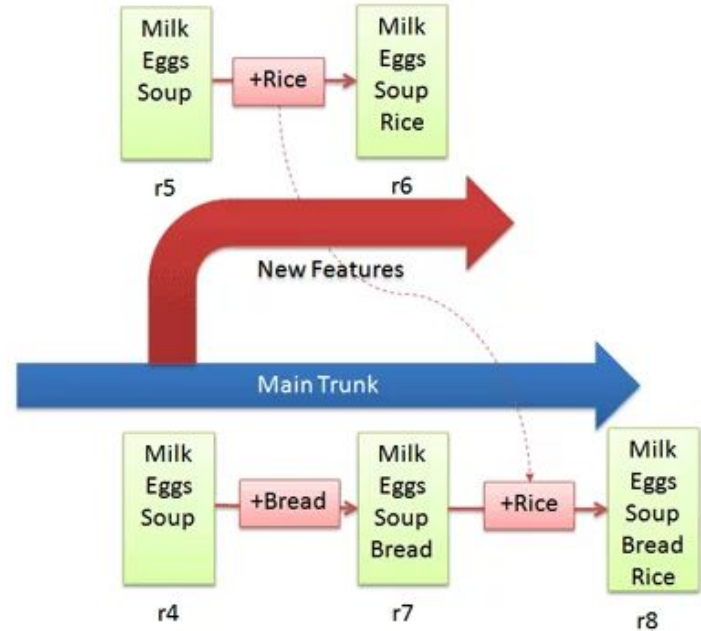
Checkout and Edit



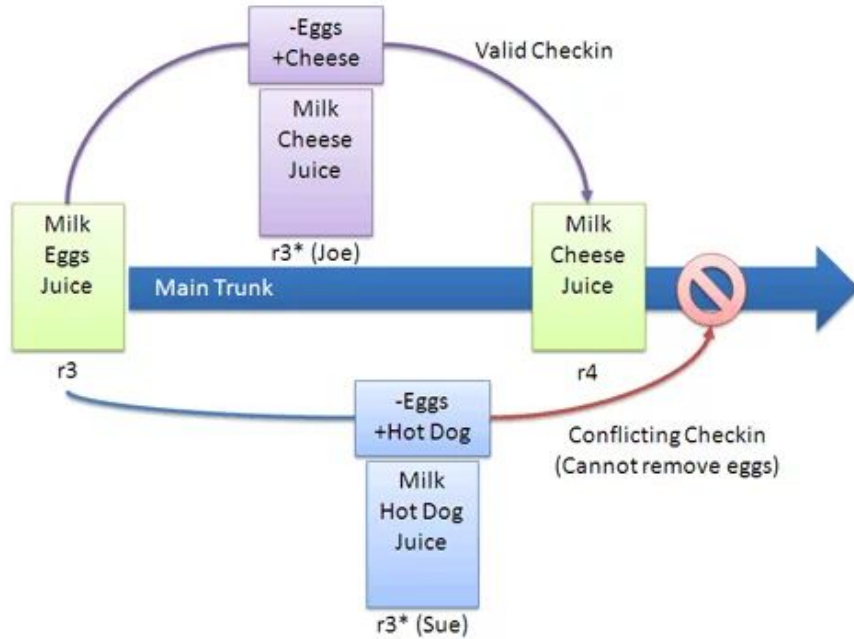
Branching



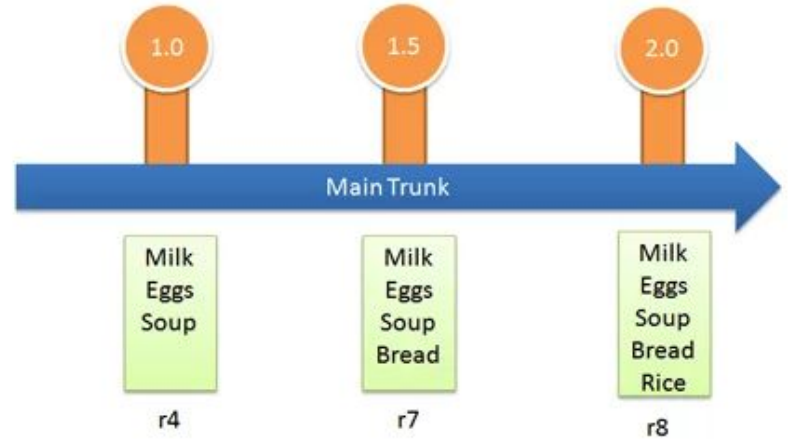
Merging



Conflicts



Tagging



More Realistic Example Auto-Merge

```
int main(){
    char* ptr;
    FILE* data;
    char buf[100];
    ....some code....
    data = fopen("data.txt", "r");
    if(fgets(buf,100,data) == NULL)
        return 1;
    printf("Read: %s", buf);
    fclose(data);
    data = fopen("data.txt", "w");
    fwrite("juice\n", 6, 1, data);
    fclose(data);
    return 0;
}
```

```
int main(){
    char* ptr;
    FILE* data;
    char buf[100];
    ....some code...
    data = fopen("data.txt", "r");
    if(fgets(buf,100,data) == NULL)
        return 1;
    printf("Read: %s", buf);
    fclose(data);
    data = fopen("data.txt", "w");
    fwrite("soup\n", 7, 1, data);
    fclose(data);
    return 0;
}
```

What Could Go Wrong?



More Realistic Example Needing Manual Resolution

```
/* original */
void display(char* str){
    write(1, str, strlen(str));
}
int main(){
    char* msg_ok = "Ok!\n";
    char* user = calloc(256, 1);
    char* pass = calloc(256, 1);
    strncpy(user, "default", 8);
    strncpy(pass, "default", 8);
    ...some code...
    write(2, msg_ok, strlen(msg_ok));
    display(user);
    return 0;
}
```



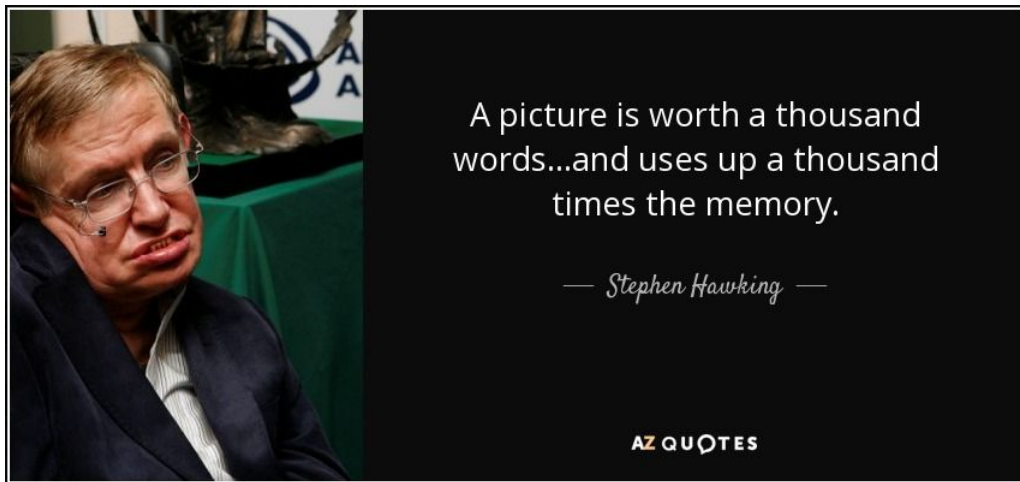
Why?

```
/* concurrent change */
void display(char* str){
    write(1, str, strlen(str));
}
int main(){
    char* msg_ok = "Ok!\n";
    char* user = calloc(256, 1);
    char* pass = calloc(256, 1);
    strncpy(user, "default", 8);
    strncpy(pass, "default", 8);
    .... some code...
    read(0, user, 255);
    read(0, pass, 255);
    write(2, msg_ok, strlen(msg_ok));
    display(user);
    return 0;
}
```

```
/* concurrent change */
void display(char* str){
    write(1, str, strlen(str));
}
int main(){
    char* msg_ok = "Ok!\n";
    char* user = calloc(256, 1);
    char* passwd = calloc(256, 1);
    strncpy(user, "default", 8);
    strncpy(passwd, "default", 8);
    ...some code...
    read(0, passwd, 255);
    write(2, msg_ok, strlen(msg_ok));
    display(user);
    return 0;
}
```

Agenda

1. Miniproject Overview
2. Version Control Overview
3. Github demo: Bin Choi



Bin Choi Github Demo

[demo slides](#)



Learn More

Video: beginner crash course [here](#), intermediate tutorial [here](#), advanced tutorial [here](#)

Text: Git documentation [here](#), Github documentation [here](#)



Next Week

Finish anything we didn't get to today

Basic bug finding as needed for
[Miniproject Part 1](#), with demos

Services and Libraries and Apps, Oh My!



Are you on the Waitlist?

Student Services Online ► Wait List

As of this writing, the class is "full", but typically many slots open up during the program change period (between now and September 13)

As slots become available, I will admit students on the waitlist who responded to the "[background check](#)" in the order the responses were received. Students who do not respond will never be admitted to the class

The screenshot shows a web browser window titled "Wait List Quick Guide_Student.pdf (1 page)". It contains a message from the instructor and a table of waitlist classes. Numbered callouts are placed on the page: 1 points to the confirmation message, 2 points to the instructor's message, 3 points to the 'Delete' button in the waitlist table, and 4 points to the 'Wait List Statures' section.

Message from the instructor:
No office hours are scheduled 11am-1pm.
To join this class you must submit a writing sample to my email below:
real123@columbia.edu

Add or Check Class Status
Class: [] [Get/Add Class]

Select	Class/Title	Instructor	Points/Option	Call #	Day	Time/Location	Start/End
☐	CHEM 343C sec 001 ESSENTIAL DATA STRUCTURES	Jae Lee	4.00 Letter Grade	67000	Tu Th	5:40pm-6:55pm BTBA BTBA	09/03/2013 12/09/2013

Wait 2013's Wait List
You are not registered for the classes below. You may register for classes that become available to you directly from your wish list.

Delete	Class/Title	Instructor	Points/Option	Day	Time/Location	Start/End	Status
☐	CHEM 305C sec 001 PHYSICAL ANALYTICAL LABORATORY I	Luis Aida	4.00 Letter Grade	Tu	12:00pm-6:00pm BTBA BTBA	09/03/2013 12/09/2013	Waitlist Available
☐	CHEM 343C sec 002 ORGANIC CHEMISTRY I-LECTURES	Luis Campos	3.50 Letter Grade	Tu Th	2:40pm-3:55pm BTBA BTBA	09/03/2013 12/09/2013	Waitlist Pending
☐	FREN 1102W sec 003 ELEMENTARY FRENCH II	Berence Baudry	4.00 Letter Grade	Mo Tu We Th	11:40am-12:45pm BTBA BTBA	09/03/2013 12/09/2013	Waitlist Denied
☐	WRIT 2001R sec 001 INTERMEDIATE FICTION WORKSHOP	Rachel Sherman	3.00 Letter Grade	We	11:00am-12:50pm BTBA BTBA	09/03/2013 12/09/2013	Waitlist Pending

1 Confirmation message appears when a course is added to the Wait List.
2 A Message from the instructor may appear, along with additional instructions (if applicable).
3 Delete courses that you are no longer interested in tracking via the Wish List.
4 Wait List Statures:
+ **Available** - Wait List is available for this course. Click **Join List** to request registration in the course.
+ **Denied** - Student has been denied entry to the course. Click **Leave List** to remove course from List.

Ask Me Anything

